

Lab 06

Object Oriented Programming

BS DS/CY Fall 2025 Morning/Afternoon

Instructions:

- Indent your code properly.
- Use meaningful variable names and follow standard naming conventions.
- Use clear and meaningful prompts/labels for all program input and output.
- Make sure that there are NO dangling pointers or memory leaks in your program.
- Do not discuss your problems with fellow students. If you face any difficulty in understanding, consult your TAs.

Task # 1

Create a class **Employee** that has private member variables to store each of the following:

- **ID** (an integer to uniquely identify each employee)
- **Name** (a char array to hold the employee's name)
- **Dept. Name** (a char array to hold the employee's department name)

*Note: The sizes of the two c-strings MUST be declared as **named constants** before the start of the class declaration as follows:*

```
// Named constants  
const int NAME_SIZE = 50;  
const int DEPT_SIZE = 50;
```

Now, carry out the following tasks in the given order:

- i.) Implement a **default constructor** for **Employee** class that assigns 0 to the ID, and null to the name and department.
- ii.) Implement an **overloaded constructor** for **Employee** class that accepts the following 3 values (in the given order) as arguments and assigns them to the appropriate member variables: employee's ID, employee's name, and department name.
- iii.) Implement an **overloaded constructor** for **Employee** class that accepts the following 2 values (in the given order) as arguments and assigns them to the appropriate member variables: employee's ID and employee's name. The department name should be assigned null.
- iv.) Implement the getter and setter functions for each member variable of the **Employee** class.
- v.) Implement a member function **display** of the **Employee** class which should neatly display all attributes of an employee on screen. This function should use the getter functions to get the values of all attributes of an employee.
- vi.) In the main function, **dynamically allocate an array of Employee** objects. Your program should ask the user about the size of the array and then allocate the array dynamically. Then, your program should ask the user to enter the values of all attributes for each **Employee**.

After that, your program should display the details of each object on screen (by calling the **display** function for each **Employee**).

Task # 2

Design an ADT 'Set' whose objects should be able to store an integer set. Size of the set will be given by the user.

Data Members:

- **int *data;** /* pointer to an array of integers
which will be treated as set of integers */
- **int noOfElements;** // number of elements in the Set
- **int capacity;** /* maximum possible number of elements
that can be stored in the Set */

Supported Operations:

The class 'Set' should support the following operations

1. **Set (int cap = 5);**
Check if cap is valid; if cap <= 0 set capacity to 5, otherwise set it to cap. Dynamically allocate the array (data = new int[capacity]) and set noOfElements = 0.
2. **Set(const Set & ref)**
Copy capacity and noOfElements from ref, allocate a new array of the same size, and copy all elements from ref.data into the new array (deep copy).
3. **~Set()**
Free the dynamically allocated memory.
4. **void insert (int element);**
Check that the set is not full and the element does not already exist; if valid, store it in the set.
5. **void remove (int element);**
Search the element in the array; if found, replace it with the last element and decrease noOfElements.
6. **int getCardinality()**
returns the number of elements in the set .
7. **Set calcUnion (Set & s2)**
returns a new Set object which contains the union of 's2' set and calling object set .
8. **Set calcIntersection (Set & s2)**
returns a new Set object which contains the inter section of 's2' set and calling object set .
9. **Set calcSymmetricDifference (Set & s2)**
returns a new Set object which contains the symmetric Difference of 's2' set and calling object set .
Where symmetric difference is: $A \Delta B = A \cup B - A \cap B$

Home Task

Create a class **Student** that has private member variables to store each of the following attributes:

- **Roll Number:** An integer variable that holds the student's roll number
- **Name:** A c-string that holds the student's name (assume that max length of name is 40)
(Note: Declare the size of the c-string as a named constant before the start of class declaration)
- **Number of Quizzes:** An integer to store the number of quizzes taken by the student
(Note: Number of quizzes can be different for each student)
- **Marks:** An int* through which you will allocate an array (of appropriate size) to store the marks obtained by the student in different quizzes
- **Total Marks of each Quiz:** An integer to store the total marks of each quiz.

Now, carry out the following tasks in the given order:

- i.) Implement a **Default Constructor** for **Student** class in which roll number should be given the value 0, name should be initialized to null, marks should be initialized as a null pointer, number of quizzes should be initialized to 0, and total marks of each Quiz should be initialized to 10.
- ii.) Implement an **Overloaded Constructor** for **Student** class that accepts 4 arguments: student's roll number, student's name, number of quizzes taken by the student, and total marks of each quiz. The values supplied in the arguments should be used to initialize the corresponding member variables, and dynamically allocate memory through the pointer marks. Each element of the marks array should be initialized to 0.
- iii.) Implement the **Destructor** for **Student** class which should deallocate all dynamically allocated memory (if any).
- iv.) Implement a public member function **getInputFromUser** of the **Student** class, which should ask the user to enter the following 4 attributes: Roll No., Name, No. of Quizzes taken by the student, and Total marks of each quiz. After storing this data in appropriate member variables, this function should allocate the Marks array. If the Marks array has been previously allocated make sure to deallocate it first (see the description of overloaded constructor above in step (ii)). After that this function should call the following private member function **inputMarks** to input the marks of each quiz (see next step).
- v.) Implement a **private** member function **inputMarks** of the **Student** class, which should ask the user to enter the marks of all quizzes (one-by-one) and store these marks in the array inside the calling Student object. Also perform **input validation** on the quiz marks entered by the user. Marks of each quiz should be greater than or equal to 0 and less than or equal to the total marks of the quiz.
- vi.) Implement a public member function **display()** of the **Student** class which should display the roll number, name, marks obtained by the student in different quizzes, and the **average marks** obtained by the student.
- vii.) Implement the **Copy Constructor** for **Student** class which should make a *deep copy* of the Student object whose reference it will receive as a parameter.
- viii.) Implement a **global function void printStudent (Student)**. This function should take a Student object which is passed by value into it. This function should display the details of that Student object on screen by calling its display function. There will be only a single statement in the body of this function ☺. The purpose of this function is to test the implementation of Copy Constructor which you implemented above in step (viii).
- ix.) Write a driver program (**main function**) which should demonstrate all of the above functions.